Comprehensive Guide to Multi-Role Authentication in Laravel

Author: Manus AI **Date:** January 22, 2026

Introduction

This guide provides a detailed, step-by-step implementation of a multi-role authentication system in a Laravel application. It is designed for developers, particularly interns, who need to understand the core concepts of Laravel's default authentication and how to extend it to manage different user types, such as **Super Admin**, **Admin**, **Sub Admin**, and **Standard User**.

The methodology is based on the principles demonstrated in the provided video, which uses a simple **role ID** stored in the database and **custom middleware** to control access.

Prerequisites

Before starting, ensure you have the following installed:

- **PHP** (version 8.1 or higher)
- **Composer**
- **Laravel Installer**
- A **Database** (e.g., MySQL, SQLite)

Step 1: Setting Up the Default Laravel Authentication

We will use **Laravel Breeze**, the official and simplest starter kit for implementing Laravel's authentication features.

1.1. Create a New Laravel Project

Use Composer to create a new project.

```
composer create-project laravel/laravel multi-role-app
cd multi-role-app
```

1.2. Install Laravel Breeze

Install the Breeze package and scaffold the authentication views.

```
composer require laravel/breeze --dev
php artisan breeze:install
```

When prompted, choose **Blade** for the stack.

1.3. Run Migrations

Ensure your database is configured in the `.env` file, then run the migrations to create the default `users` and other tables.

```
php artisan migrate
```

At this point, you have a fully functional default login and registration system.

Step 2: Modifying the Database for Roles

To support multiple roles, we need to add a `role` column to the `users` table.

2.1. Create a New Migration

We will create a new migration to add the column.

```
php artisan make:migration add_role_to_users_table --table=users
```

2.2. Update the Migration File

Open the newly created migration file (e.g., `database/migrations/YYYY_MM_DD_HHMMSS_add_role_to_users_table.php`) and update the `up` and `down` methods.

File: `database/migrations/YYYY_MM_DD_HHMMSS_add_role_to_users_table.php`

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            // Add the 'role' column with a default value of 0 (Standard
User)
            $table->tinyInteger('role')->default(0)->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

2.3. Run the New Migration

```
php artisan migrate
```

2.4. Define Role Constants in the User Model

For clarity and maintainability, define constants for each role in the `User` model.

File: `app/Models/User.php`

```php
<?php

namespace App\Models;

// ... other imports

class User extends Authenticatable
{
    // Define Role Constants
    public const ROLE_USER = 0;
    public const ROLE_SUPER_ADMIN = 1;
    public const ROLE_SUB_ADMIN = 2;
    public const ROLE_ADMIN = 3;

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'name',
        'email',
        'password',
        'role', // Add 'role' to fillable
    ];

    // ... rest of the file remains the same
}
```

Step 3: Role-Based Redirection After Login

The default redirection after a successful login is handled by the `RedirectIfAuthenticated` middleware and the `RouteServiceProvider`. However, for a multi-role system, we need a central point to decide where to send the user.

3.1. Modify the `LoginController` (or `AuthenticatedSessionController`)

In a Laravel Breeze installation, the login logic is in `app/Http/Controllers/Auth/AuthenticatedSessionController.php`. We will override the default redirection logic.

File: `app/Http/Controllers/Auth/AuthenticatedSessionController.php`

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use App\Providers\RouteServiceProvider;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;
use App\Models\User; // Import User model

class AuthenticatedSessionController extends Controller
{
    // ... (rest of the file)

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        // --- START: Custom Role-Based Redirection Logic ---
        $user = Auth::user();

        switch ($user->role) {
            case User::ROLE_SUPER_ADMIN:
                return redirect()-
>intended(RouteServiceProvider::SUPER_ADMIN_DASHBOARD);
            case User::ROLE_SUB_ADMIN:
                return redirect()-
>intended(RouteServiceProvider::SUB_ADMIN_DASHBOARD);
            case User::ROLE_ADMIN:
                return redirect()-
>intended(RouteServiceProvider::ADMIN_DASHBOARD);
            case User::ROLE_USER:
            default:
                return redirect()->intended(RouteServiceProvider::HOME);
        }
        // --- END: Custom Role-Based Redirection Logic ---
    }
```

```
    // ... (rest of the file)
}
```

3.2. Define New Dashboard Routes in `RouteServiceProvider`

We need to define the new dashboard constants in `app/Providers/RouteServiceProvider.php`.

File: `app/Providers/RouteServiceProvider.php`

```php
<?php

namespace App\Providers;

use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Foundation\Support\Providers\RouteServiceProvider as
ServiceProvider;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Facades\Route;

class RouteServiceProvider extends ServiceProvider
{
    /**
     * The path to your application's "home" route.
     *
     * Typically, users are redirected here after authentication.
     *
     * @var string
     */
    public const HOME = '/dashboard';

    // --- START: Custom Dashboard Constants ---
    public const SUPER_ADMIN_DASHBOARD = '/super-admin/dashboard';
    public const SUB_ADMIN_DASHBOARD = '/sub-admin/dashboard';
    public const ADMIN_DASHBOARD = '/admin/dashboard';
    // --- END: Custom Dashboard Constants ---

    // ... (rest of the file)
}
```

Step 4: Creating Role-Specific Middleware

Middleware is the key to authorization. It checks the user's role before allowing access to a route.

4.1. Create Middleware Files

Create four new middleware files.

```
php artisan make:middleware IsSuperAdmin
php artisan make:middleware IsSubAdmin
php artisan make:middleware IsAdmin
php artisan make:middleware IsUser
```

4.2. Implement Middleware Logic

The logic for each middleware is nearly identical: check the authenticated user's role and redirect if it doesn't match.

File: `app/Http/Middleware/IsSuperAdmin.php`

```php
<?php

namespace App\Http\Middleware;

use App\Models\User;
use Closure;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Symfony\Component\HttpFoundation\Response;

class IsSuperAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (Auth::check() && Auth::user()->role == User::ROLE_SUPER_ADMIN) {
            return $next($request);
        }

        // Redirect to a safe place, like the login page or their own
dashboard
        return redirect('/login')->with('error', 'Unauthorized access.');
    }
}
```

File: `app/Http/Middleware/IsSubAdmin.php` (Change `User::ROLE_SUPER_ADMIN` to `User::ROLE_SUB_ADMIN`)

```php
// ... (same structure as IsSuperAdmin.php)
class IsSubAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (Auth::check() && Auth::user()->role == User::ROLE_SUB_ADMIN) {
            return $next($request);
        }
        return redirect('/login')->with('error', 'Unauthorized access.');
    }
}
```

File: `app/Http/Middleware/IsAdmin.php` (Change `User::ROLE_SUPER_ADMIN` to `User::ROLE_ADMIN`)

```php
// ... (same structure as IsSuperAdmin.php)
class IsAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (Auth::check() && Auth::user()->role == User::ROLE_ADMIN) {
            return $next($request);
        }
        return redirect('/login')->with('error', 'Unauthorized access.');
    }
}
```

File: `app/Http/Middleware/IsUser.php` (Change `User::ROLE_SUPER_ADMIN` to `User::ROLE_USER`)

```php
// ... (same structure as IsSuperAdmin.php)
class IsUser
{
    public function handle(Request $request, Closure $next): Response
    {
        if (Auth::check() && Auth::user()->role == User::ROLE_USER) {
            return $next($request);
        }
        return redirect('/login')->with('error', 'Unauthorized access.');
    }
}
```

4.3. Register Middleware in `Kernel.php`

Register the new middleware in the `$routeMiddleware` array in `app/Http/Kernel.php` so they can be used in routes.

File: `app/Http/Kernel.php`

```php
// ... (rest of the file)

    /**
     * The route middleware aliases.
     *
     * These may be used to conveniently group middlewares and apply them to
routes.
     *
     * @var array<string, class-string|string>
     */
    protected $routeMiddleware = [
        // ... existing middleware

        // --- START: Custom Role Middleware ---
        'isSuperAdmin' => \App\Http\Middleware\IsSuperAdmin::class,
        'isSubAdmin' => \App\Http\Middleware\IsSubAdmin::class,
        'isAdmin' => \App\Http\Middleware\IsAdmin::class,
        'isUser' => \App\Http\Middleware\IsUser::class,
        // --- END: Custom Role Middleware ---
    ];
}
```

Step 5: Defining Role-Specific Routes and Dashboards

Now we define the protected routes that use the new middleware.

5.1. Update `routes/web.php`

We will create separate route groups for each role.

File: `routes/web.php`

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

/*
|--------------------------------------------------------------------------
| Web Routes
|--------------------------------------------------------------------------
|
| Here is where you can register web routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "web" middleware group. Make something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

// --- START: Role-Specific Dashboards ---

// 1. Super Admin Routes
Route::middleware(['auth', 'isSuperAdmin'])->prefix('super-admin')-
>group(function () {
    Route::get('/dashboard', function () {
        return view('super-admin.dashboard');
    })->name('super-admin.dashboard');
    // Add other Super Admin routes here
});

// 2. Sub Admin Routes
Route::middleware(['auth', 'isSubAdmin'])->prefix('sub-admin')-
>group(function () {
    Route::get('/dashboard', function () {
        return view('sub-admin.dashboard');
    })->name('sub-admin.dashboard');
    // Add other Sub Admin routes here
});

// 3. Admin Routes
Route::middleware(['auth', 'isAdmin'])->prefix('admin')->group(function () {
    Route::get('/dashboard', function () {
        return view('admin.dashboard');
    })->name('admin.dashboard');
```

```
    // Add other Admin routes here
});

// 4. Standard User Routes (Default Laravel Breeze Dashboard)
Route::middleware(['auth', 'isUser'])->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard'); // Uses the default Breeze dashboard view
    })->name('dashboard');
    // The default Breeze profile routes can remain here
    Route::get('/profile', [ProfileController::class, 'edit'])-
>name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])-
>name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])-
>name('profile.destroy');
});

// --- END: Role-Specific Dashboards ---

require __DIR__.'/auth.php';
```

5.2. Create Dashboard Views

Create the necessary directories and simple view files to confirm the redirection and access control are working.

File: `resources/views/super-admin/dashboard.blade.php`

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Super Admin Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    You are logged in as a **Super Administrator**!
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

File: `resources/views/sub-admin/dashboard.blade.php`

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Sub Admin Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    You are logged in as a **Sub Administrator**!
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

File: `resources/views/admin/dashboard.blade.php`

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Admin Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    You are logged in as an **Administrator**!
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

(Note: The Standard User will use the existing `resources/views/dashboard.blade.php`.)

Step 6: In-View Feature Authorization

The final step, as mentioned in the video, is controlling the visibility of menu items or features within a shared layout.

6.1. Example: Controlling Navigation Links

If you have a shared navigation bar (e.g., in `resources/views/layouts/navigation.blade.php`), you can use the `Auth::user()->role` property to conditionally display links.

File: `resources/views/layouts/navigation.blade.php` (Snippet)

```
<!-- ... existing navigation code ... -->

<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
    <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
        {{ __('User Dashboard') }}
    </x-nav-link>

    @if (Auth::user()->role == \App\Models\User::ROLE_SUPER_ADMIN)
        <x-nav-link :href="route('super-admin.dashboard')" :active="request()->routeIs('super-admin.dashboard')">
            {{ __('Super Admin Panel') }}
        </x-nav-link>
    @endif

    @if (Auth::user()->role == \App\Models\User::ROLE_ADMIN || Auth::user()->role == \App\Models\User::ROLE_SUPER_ADMIN)
        <x-nav-link :href="route('admin.dashboard')" :active="request()->routeIs('admin.dashboard')">
            {{ __('Admin Panel') }}
        </x-nav-link>
    @endif
</div>

<!-- ... rest of the navigation code ... -->
```

This ensures that the “Super Admin Panel” link is only visible to users with a role ID of `1`, and the “Admin Panel” link is visible to users with role IDs `1` or `3`.

Summary of Implementation Flow

The entire multi-role authentication flow is achieved through three simple mechanisms:

1. **Database Role ID:** A single column (`role`) determines the user’s identity.
2. **Login Redirection:** A `switch` statement in the login controller redirects the user immediately after login based on their role ID.
3. **Middleware Protection:** Custom middleware blocks unauthorized access to role-specific routes by checking the role ID.

This approach is simple, effective, and directly implements the strategy demonstrated in the video, making it easy for your interns to understand and maintain.

References

[1] Laravel Breeze Documentation [2] Laravel Middleware Documentation [3] Laravel Authentication Documentation [4] YouTube Video: How to Authenticate Multiple Users in Laravel